

https://github.com/msoud2509/5105_project_3/tree/main

System Architecture

Our approach, at a high level, is quite similar to the docker approach as described in the project writeup. We have a single controller that interacts directly with the client, then the controller will route requests in a round-robin system to the different service nodes.

Each service node is attached to a “local” storage node, which is the associated storage node that they solely interact with. The controller manages the associations and will initially assign them, but once the service node learns of its assignment, it knows to only interact with its assigned storage node.

What does your service tier actually do?

The service node essentially just forwards the request to the associated storage node, and the storage node will actually execute the data operation. Other than forwarding, the main capability of the service nodes will be to maintain auction sessions for clients. The main extra feature that comes with auction sessions is that for each item that is being bidded on in the sessions, every few seconds the service node will poll for updates, ensuring that

Because each service node will maintain their sessions individually, the controller will need to maintain a registry of auction sessions separately, to ensure that requests via auction sessions are correctly routed, which is independent of the round robin routing.

Implementations such as replication, synchronization, failure handling, are described below.

Replication Strategy

Since service nodes are basically stateless (except for auction session management), they can be directly replicated when scaled.

Storage nodes are replicated based on the entirety of the sample data. So on startup, we have 3 nodes that each contain the entirety of the sample data. The node with the largest index, 2 in this case, becomes the primary.

As for replication, we use a primary-backup model, where the controller manages who becomes the primary, and lets all of the storage nodes know. The synchronization strategy provides more detailed implementation information.

Why did you choose primary-backup or quorum?

- While primary backup is more intuitive conceptually, there certainly were implementation complexities. The forward and broadcast logic was quite difficult to coordinate. However, primary backup does allow for a more realistic setup between service and storage

nodes, where things in real distributed systems, things like geographical setup often means that it's better for service nodes to be attached to specific storage nodes. So primary backup gives both easier conceptual visualizations while also mimicking certain realistic situations.

We have the controller directly doing health checks and handling failure rollover.

Synchronization Approach

We implemented a primary-backup system where service nodes write to their respective storage nodes. A read operation is quite simple, the associated storage node just returns their value. But in the case of a write, if the storage node is not the primary, then the node will forward the request to the primary, and then the primary will broadcast the request to all other storage nodes, ensuring that the replicas remain consistent. Because the controller doesn't return success until the data has been properly replicated (written to node won't return success until the primary returns success that it successfully replicated to all nodes), it is a strong consistency model, ensuring that each read will read from the latest *completed* write.

What operations require the strongest consistency?

- Really only the update operation requires strong consistency, as we want to be sure that the current price is correct for subsequent bidding logic, as well as you always want the latest read. Placing a bid doesn't really matter in terms of strong consistency, bids can be out of order, the highest bid will always win anyways regardless of order.

Failure Handling

In the event of a failure (either the controller sees a timeout on a request to the storage node, or the storage node fails a health check), the controller will mark the storage node and associated service nodes as unhealthy (want to make sure that we don't route requests to service nodes with bad storage node associations). Then, the controller will select a new primary. And when it does so, it will spin up a new storage node in an attempt to replace the old one. Once the new storage node is healthy, the controller will begin to start routing traffic back to the associated service nodes, as the service nodes will begin forwarding to this new storage node that took the old ones place.

How does a replacement replica catch up after failure?

- Once a new storage is created via the controller, the controller will tell the primary to replicate its data to the new node (similar to how it replicates its data on startup).

Autoscaling Policy

How does a new service node join the system?

A new service node joins the system via the controller, the controller will create it via docker sdk, then will assign it a storage node (via round robin), and will add the service node to the registry, thereby beginning to route traffic to it

What event or metric triggers scaling?

The event that triggers scaling is the request rate. Once the request rate exceeds 5 requests per second(req/s) another service node will be created in order to handle some of the requests. The request rate is calculated in the controller and then periodically sent to the scaling manager to scale up or down the number of service nodes.

Workload & Eval Approach

Read-heavy test

We built an evaluation_system.py script to track some evaluation metrics. The read latency was around 43.71 ms. This is a good baseline meaning that getting the item was very fast even for a sizable number of workers. The read throughput is 684.37ms. This means that for the three node tier it is consistently sending and receiving read requests at a fast rate. This is a sign of a system that can take heavy read loads without scaling. The number of requests was 1000 and the number of workers was 10.

Write-heavy test

When testing the writes in the system which involve bidding and auctions the first time testing it around a flat \$5000 dollar bid was placed on an item which was starting at \$48.24. Since there were many bids coming in at once to the system, it accepted the first bid which was \$5000 dollars and rejected all the other bids which were at the same price. This shows the system handles race conditions as even if multiple people were to bid at once only the first one wins, given that the price that everyone is bidding at is the same(\$5000).

After increasing the number of requests to 10000, increasing the number of workers to 30 and adding a fluctuation in the bid price sent, the average latency was 39.12ms and throughput was 764.78 req/s. One thing to note is that this latency includes the replication penalty, the time it takes the primary storage node to send to backups and receive acknowledgements from them. The first request to reach the storage node and set the new price is the winner. All the other requests that were lower than the winner's bid are stale and aren't able to win the bid. Moreover it proves this system is linearizable such that it behaves like there is only one storage node, whereas in reality there are 3 storage nodes.

Auto-scaling test

The next test is the main one. When increasing the number of requests to 30000 and increasing the number of workers to 40, the system was forced to add new service nodes. The latency was around 63.06ms and the throughput was 632.76 req/s. The application is required to auto scale after it reaches a threshold of 5req/s per storage node.

Major Tradeoffs & Lessons Learned

We decided to implement a strong consistency pattern, just because that was most natural in a primary-backup system. This does sacrifice latency a bit, but because it's only 3 storage nodes, and it makes sure that the read gets the latest write as quickly as possible, this is an acceptable tradeoff.

What tradeoffs did your design make between simplicity and realism?

- Number of requests are generally not a good indicator of needing to scale, but because this is run purely on local, and service nodes are the only ones scaling, storage nodes and especially the controller could very well be the bottleneck if we measured latency for example, so it makes sense to do something like number of requests in a given timeframe so the system will keep working properly, but still somewhat reliably measures overall traffic.
- However, not everything was made with the idea of simplicity in mind. A big tradeoff for simplicity was the fact that service nodes are attached to specific storage nodes. It would be simpler for the service nodes to do a sort of round robin themselves to storage nodes, as the coupling requires extra logic, but it is more realistic that service nodes route to certain storage nodes. For example, it doesn't make any sense for a server in the US to route to a db in China, when the data can be found in a db somewhere else in the US.

Lessons Learned

- Design decisions should be considered with a picture that is as specific as possible. A concept might seem simpler in practice but as you actually implement and figure out what other components of a system are affected, it might actually be more complicated than you originally thought.
- These theory heavy concepts that we discuss in class are easier said than done. Concepts like primary backup and consistency are quite easy to understand, but implementing them and converting ideas to code, and handling edge cases and integrating them with the rest of the system can be quite challenging (there's a reason why there are so many abstractions in software).